

Technical Appendix: Controller-Preserving Update Synthesis

Anonymous Author(s)

This technical appendix accompanies the paper. The main paper gives the complete Cell contract, update-game specification, contract correspondence, the continuation proof, synthesis guarantees and certificate checks. This document supplies the detailed source-history definition, remaining correctness proofs, residual-soundness formalization, full tables and further history and execution arguments. References to the main paper use its section and algorithm numbers. S1–S5 remain a separate supplement.

A Source Histories and Synthesis Correctness

Source histories without a global game. Fix the eligible local contract of Section 3. A history is $\ell = z(a_1, o_1) \cdots (a_n, o_n)$, starting at a reachable old closed-loop tuple z ; o_j records the realized physical successor tuple. For each prefix, reconstruct installed versions $v_i(\ell)$ and local states $s_i(\ell)$ by the recurrences below. Let $D(\ell)$ be its executed update commands. Initially $D = \emptyset$, every version is old and local states are those in z . Old monitor memory is inherited from z , each interval monitor starts at its supplied initializer, and every new monitor is inactive. The old endpoint controller imposes no restrictions after entry.

Enabled local events and outcomes. For an ordinary label a , let $J_a(\ell) = \{i : a \in \Sigma_{e_i}^{v_i(\ell)}\}$. It is enabled exactly when $J_a(\ell) \neq \emptyset$ and every $i \in J_a(\ell)$ has a nonempty local a -successor set. Every tuple choosing one such successor per participant, and retaining all other local states, is an outcome. This test ignores monitors, safety and targets. The prefix is UC-quiescent exactly when no enabled ordinary label is uncontrollable, including labels with unsafe monitor outcomes. A command a may occur only at a UC-quiescent prefix, with $a \notin D(\ell)$ and all its declared predecessors in $D(\ell)$.

An ordinary event preserves versions and D . A transfer ρ_i additionally requires $v_i = \text{old}$ and chooses any $s'_i \in g_i(s_i)$, changes only that installed component to (new, s'_i) , and adds ρ_i to D . An empty transfer image permits no extension. A stop or start leaves all physical states unchanged and adds itself to D ; start additionally requires the current version-tagged tuple in its initializer domain. Every enabled outcome yields a source extension, even if it violates a monitor. Thus monitors test behavior rather than disabling physical events.

Memory, boundaries and source objectives. Monitor memory follows a deterministic fold on the history. Ordinary events and transfers advance every active monitor by its transition function. A stop makes its named old monitor inactive without letting it observe that stop, and advances all other active monitors once. A start advances previously active monitors once, then installs the named new monitor's initializer value; the new monitor does not consume start again. Other inactive memories remain inactive. The executed-command set makes every boundary one-shot. A prefix is safe when every active memory is outside its error set.

Handover is allowed precisely at safe UC-quiescent prefixes with every command executed and with the physical tuple and new-monitor memories matching a supplied target; the selected target also supplies its controller state. A source policy may depend on the entire history and may disable only controllable labels. Every enabled uncontrollable, and every outcome of every permitted label, remains possible. A policy wins if all its prefixes from every old entry are safe, no non-handover prefix is a dead

end, and every maximal history reaches handover after finitely many events. Neither fairness nor a choice of local outcomes is available to the policy. These operational clauses precede the global-state construction and are the source obligations used in Proposition 3.1.

Source histories describe supplied monitor behavior. They do not infer the semantic activation-history sets H_r in RS or prove those sets complete for an application. The scope-check populations and their limits are unchanged. The main paper states Proposition 3.1 for these source operations.

Proof of Proposition 3.1.

Proof. The entry map retains components and old-monitor memory and installs interval states. Current versions and states determine all ordinary product outcomes and uncontrollables, including unsafe outcomes. Pending commands determine one-shot and precedence guards; local relations and initializers determine transfer and boundary outcomes. Equal images thus have equal next choices and outcomes. Monitor folds coincide with all four transition rules, preserving safety and targets without consuming boundaries twice. Released old-controller memory adds no constraint. Induction extends source histories and game paths in both directions. Transporting policies preserves finite and infinite outcome paths, non-goal deadlocks and goals, hence safe finite completion. The memoryless-policy lemma below then yields a state-based winner. $\square$

Lemma (memoryless policies suffice). If any history-dependent policy wins this finite game, a state-based policy wins. *Proof.* Starting with goals, attractor layers add safe states whose nonempty proving sets lie in previous layers. Choosing those sets gives a state-based policy with decreasing layer index. Outside the fixed point, the environment can keep an outcome outside after every history, or force an unsafe state or deadlock, defeating any history-dependent policy (S2).

Lemma (monitor intervals). Along every update prefix, each active monitor is its inherited or initialized state advanced over exactly the labels in its declared active interval. *Proof.* Entry inherits old states and initializes interval monitors. Ordinary events and transfers advance all active monitors once. Stop removes its monitor before observation, while others advance. Start advances existing monitors and installs the new one after its boundary. Pending commands enforce one-shot execution and consistent versions/activity. Induction gives the stated interval; safety makes it non-error, and goal matching preserves new-monitor states into the endpoint.

Lemma A.1 (UC priority). *Disabling controllable events at UC-enabled states preserves the existence of a policy for finite safe completion.*

Proof. A winning policy must handle every UC outcome. Disabling optional controls removes runs and creates no deadlock while a UC event remains enabled. Every surviving maximal run was already a maximal run of that policy, so it still completes safely. Conversely, the reduced policy is admissible in the original game because controllers may disable those events. $\square$

Lemma A.2 (Search coverage). *At loop boundaries, every state in X has all UC candidates queried; every expanded unproved quiescent safe non-goal with candidates left is in Retry; and every discovered safe non-goal outside X is in Open.*

Proof. Initialization at entry states establishes coverage. Discovery queues each new safe non-goal; only expansion removes it, after querying all UC candidates. At quiescence, expansion and retries consume the cursor through a nonempty bucket or exhaustion, retaining every nonwinner with candidates left. Promotion removes only winners. These are the only changes to those sets; finite propagation drains before either return. $\square$

Proof of Theorem 4.1.

Proof. Termination. The finite input induces finitely many states and candidates. Each state is discovered and expanded at most once; each retry consumes a new candidate, and each promotion adds a state to W once. Reverse incidences are processed on those promotions, so every worklist drain is finite. Eventually all entries win or both exploration queues empty.

Soundness. Induction on promotions gives complete successor sets with smaller retained ranks: expansion supplies all uncontrollables, or the selected nonempty controllable bucket supplies all outcomes. A queried bucket never changes, and ranks are fixed on first promotion, so this evidence persists. If all entries win, their policy closure is safe and every non-goal has a successor. Strict decrease of the natural-number rank excludes infinite runs; nonempty successors exclude non-goal sinks. Thus every maximal run reaches a goal within its initial rank many events. Goal recognition uses modeled uncontrollable quiescence and supplies each compatible $\kappa(q)$.

The monitor-interval lemma above supplies the interval invariant; goal matching preserves components and new-monitor states into the verified endpoint.

Completeness. At failure, promotion has reached its fixed point. An entry lies in $L = V \setminus W$, and L contains no goal. By Lemma A.2, empty Open ensures every safe non-goal in L was expanded. With uncontrollables, complete expansion and exhausted propagation imply an uncontrollable outcome in L ; otherwise promotion would apply. Without uncontrollables, empty Retry gives every enabled controllable bucket, each with an outcome in L for the same reason. Against any policy, the environment keeps choosing these outcomes; permitting no event produces a deadlock. The resulting run reaches an unsafe state, deadlocks outside a goal, or remains in L forever. Each contradicts winning. Hence a winning policy rules out failure; termination then forces success.

The losing-region strategy defeats history-dependent policies after every history; success supplies a ranked state-based policy. Lemma A.1 preserves winning in the original supplied game. $\square$

Promotion cases. Main Figure 4 displays both promotion cases. Every UC outcome must win when UC is enabled. At quiescence one control bucket suffices, but all its nondeterministic outcomes must win; other control buckets may remain unqueried.

B Complete Cell Contract and Its Two Completing Paths

Table 1 in the main paper gives all component, endpoint-controller and monitor transitions, including the conventions for absent edges.

Endpoint tuples list stations A, B , controller and monitor. States h/e mean holding/empty; k, c, d mean old-safe, inspection-clear and inspection-pending; c_o, c_0, c_1 name controller states. Old reachable tuples are $x_A = (h, e, c_o, k)$ and $x_B = (e, h, c_o, k)$; new ones are $z_A = (h, e, c_0, c)$, $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Initial tuples are x_A, z_A and loadable targets are $\{z_A, z_B\}$, excluding pending inspection at y_B . The interval initializer records the holder at either all-old one-workpiece tuple. The new initializer admits all eight version-tagged one-workpiece tuples, installing d when B holds and c otherwise. The paths starting with A/B holding map to z_B/z_A at handover.

With empty-only transfers $g_A = g_B = \{(e, e)\}$ and exactly one workpiece, the two paths $\rho_B; m_B; \text{start}; \text{stop}; j; \rho_A$ from A holding and $\rho_A; m_A; \text{start}; \text{stop}; \rho_B$ from B holding have safe singleton outcomes: start installs inspection-pending if B holds and inspection-clear otherwise, stop removes the inspection ban, and both paths finish with new versions and clear inspection at matching supplied targets. Their remaining lengths $6, \dots, 0$ and $5, \dots, 0$ strictly decrease, proving finite completion from both reachable old states (13 policy states and 11 edges). Replacing the two transfer commands by one with relation $g_A \times g_B$, with endpoints and requirements fixed, requires both stations empty, but every ordinary move, inspection and requirement boundary preserves exactly one workpiece; that command can never execute, stays pending and makes every goal unreachable.

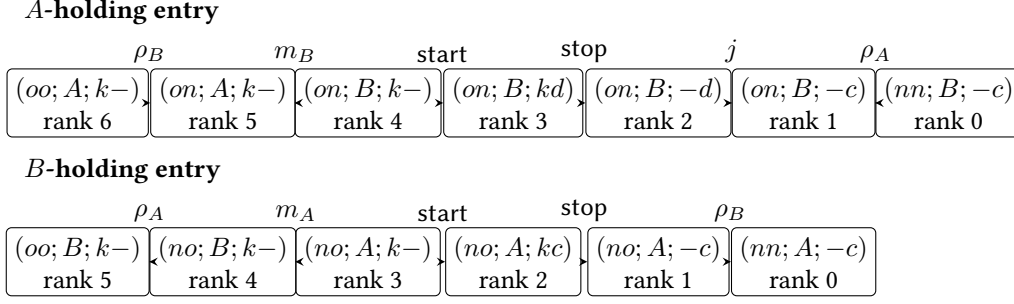


Figure A1: One policy from both entries. States list versions of A , B , the workpiece holder, and old/new monitors; the holder also determines the interval monitor. A dash means inactive. Every edge decreases the remaining-step rank. The two rank-zero states match the respective fixed-controller targets.

B.1 Residual Soundness and the Cell Reference

Residual soundness (RS) relates monitor memory to prior history. Let SafePref_r be requirement r 's prefix-closed language, and let $H_r(\xi)$ contain all admissible activation histories consistent with observed component state ξ . A new requirement's history includes start once; its initialized monitor reads only later events. Residual-sound initialization means

$$\underbrace{\{w : \eta_r^*(\iota_r(\xi), w) \notin F_r\}}_{\text{continuations accepted by the initialized monitor}} \subseteq \underbrace{\bigcap_{h \in H_r(\xi)} \{w : hw \in \text{SafePref}_r\}}_{\text{continuations safe after every admitted history}} \quad (\text{RS})$$

for every $\xi \in \text{dom } \iota_r$. Equality is exact; strict inclusion is conservative and may exclude feasible updates. An empty history set makes the right side universal. Synthesis checks initializer states and domains, but neither infers histories nor establishes their completeness.

For Cell's declared start-only reference, $H_{r_{\text{inspect}}}(\xi) = \{\text{start}\}$. Initializing at c is exact for that reference; initializing at d when B holds is conservative, forbidding departure until inspection. The complete inclusion proof is in Technical Appendix B.1. This distinction exposes a separate obligation: an exact history residual need not encode the intended duty at activation.

A Boolean event-history variable, or *fluent*, records setting/resetting labels. For an initially true fluent p , a history making p false then true and the empty history end at the same value but differ under $\Box p$.

Deriving RS in the cell. The declared pre-activation observation retains only start, so $H_{r_{\text{inspect}}}(\xi) = \{\text{start}\}$ for every admissible cell tuple. The reference monitor starts at c , unchanged by start; the right-hand side of RS is therefore its non-error continuation language from c . If B is empty, initialization at c gives equality. If B holds, initialization at d is conservative: before the first j or m_B , d additionally forbids m_A ; either j or m_B brings the two initial states to the same state. Thus every word safe from d is safe from c , while the single event m_A witnesses strict inclusion. This is a proof for the declared observation scope, not recovery of pre-activation history.

Initializing at c while B holds satisfies equality but omits the intended pending-inspection duty encoded by d . Declared histories and intended obligations are therefore separate input choices.

C Complete Benchmark Input Counts

Inputs have 1–5 components, 4–131 entries, 1–12 old/new requirements each and 3–26 commands, using PC2/R2's input log.

Table A1: The nine adapted inputs. Old/new requirements are shared across variants; interval counts are Base/R1/R2.

Input	Comp.	Entries	Old/New	Interval B/R1/R2	T	U
GSM	1	8	1/1	0/1/2	8	3
Industry	3	131	3/3	0/3/6	16	9
MetaSocket	2	4	1/1	0/1/2	3	4
PowerPlant	2	10	4/3	0/3/7	4	9
PC Arms=1	1	9	6/6	0/6/12	7	13
PC Arms=2	2	126	12/12	0/12/24	14	26
Railcab	4	22	5/7	0/7/11	11	16
Surveillance	2	112	7/7	0/7/14	16	16
Workflow	5	28	5/6	0/6/11	12	16

T counts declared transfer pairs before fluent observers; U counts transfer, stop and start commands. Entries agree across completed variants; PC Arms=2/R2 times out and its requirement counts come from its input log. Only Railcab has branching transfers. These nine inputs declare no explicit precedence; the separate cell witness requires start $\prec$ stop; R2 uses interval monitors.

D All Completed Direct-Full Exploration Pairs

The all-contract outcome summary is Table 5 in the main paper. The table below reports every completed pair.

Table A2: Absolute exploration costs for all 14 completed Direct-Full pairs. Figure A2 displays every fixed-budget contract for all four methods.

Input / variant	States (L/DF)	Queries (L/DF)	Solver s (L/DF)
GSM / Base	31 / 75	27 / 658	0.0224 / 0.0416
GSM / R1	34 / 115	44 / 706	0.0228 / 0.0450
GSM / R2	34 / 68	78 / 352	0.0201 / 0.0327
Industry / Base	272 / 282,709	3,336 / 5,058,452	0.190 / 154
Industry / R2	7,717 / 51,096	52,919 / 725,391	1.15 / 28.0
MetaSocket / Base	8 / 82	12 / 289	0.0149 / 0.0408
MetaSocket / R1	8 / 95	12 / 257	0.0136 / 0.0426
MetaSocket / R2	16 / 34	24 / 79	0.0152 / 0.0240
PowerPlant / Base	73 / 8,168	165 / 47,618	0.0463 / 1.85
PowerPlant / R1	759 / 23,686	1,927 / 66,394	0.125 / 5.51
PowerPlant / R2	1,330 / 3,594	3,118 / 13,512	0.180 / 1.01
PC Arms=1 / Base	83 / 407,019	840 / 3,842,690	0.0741 / 157
PC Arms=1 / R2	56,443 / 158,218	354,259 / 1,363,292	6.16 / 77.8
Railcab / R2	97,522 / 317,094	437,383 / 2,842,593	15.5 / 283

L: Lazy; DF: Direct-Full. All 14 WIN/WIN pairs are shown. States and successor-oracle queries are exact first-trial counts. Solver seconds are medians of five valid runs, including internal checking and excluding JVM/frontend/endpoint time, rounded to three significant digits.

E Default Exploration Order

Lazy orders stop-transfer-ordinary while stops remain, then transfer-ordinary while transfers remain, with other updates last. Otherwise, start leads only when an endpoint target matches components and current or initialized new monitors. State priority uses endpoint mismatch and pending-command counts; final ties use event type/Java labels and canonical state order. LIFO exploration precedes FIFO retries (full order in S2); it need not minimize exploration.

Scope of the work bound. Let d count discovered states, c candidate queries (including empty results), and o returned outcome incidences over those queries. Under the main paper’s indexed-predicate, output-sensitive successor and constant-time bookkeeping assumptions, Algorithm 1 admits an implementation with $O(d + c + o)$ work and storage. This counts the finite explored portion, not the potentially exponential implicit game. It excludes endpoint preprocessing, representation-dependent predicate costs and the Java implementation’s extra sorting/scanning.

Counters and charges. Intern each state once and queue its expansion once. A cursor advances monotonically through each state’s finite candidates; thus each query and each cursor/Retry transition is charged to a candidate or state. For each queried nonempty bucket, retain an outcome counter and the maximum rank of already proved outcomes. An outcome pointing to a winner updates these immediately; otherwise store one reverse incidence at its target. When a state first enters the winning set, process each of its reverse incidences once. Hence all counter decrements and maximum updates total $O(o)$.

At a state with enabled uncontrollables, combine their returned outcomes into the required universal proving set; promotion is enabled only after every UC candidate has been queried. At a quiescent state, any nonempty control bucket whose counter reaches zero can certify promotion. Recording the first such bucket needs constant work per completed bucket. Nonempty flags prevent an empty bucket from certifying progress. A state is promoted at most once; its rank is one above its proving set’s accumulated maximum, and the retained witness supports extraction.

Maintain Open, Retry and the promotion worklist by indexed membership and removable entries, so enqueue/dequeue/removal is constant time and exhaustion is detected without repeated global scans. Goal initialization and unsafe-state exclusion cost $O(d)$. The final policy traversal or losing-region extraction scans only stored states and incidences. Summing these charges proves the stated bound. The worklist computes the same monotone promotion fixed point as the abstract algorithm; this argument does not assert a linear running time for every implementation of its instruction to exhaust promotion.

Exploration containment and cost model. With indexed predicates, output-sensitive successors and constant-time bookkeeping, a counter implementation takes linear work in discovered states, queried candidates and returned outcomes (Technical Appendix E). Endpoint preprocessing is excluded; Java additionally sorts transitions and scans entries (S2). Direct-Full constructs the whole reachable game with unsafe/goal terminals; Lazy may also reach it in the worst case.

Lazy discovers a subgraph of Direct-Full: induction over queries preserves shared entries, successors and terminal predicates. Neither expands unsafe states or goals; empty queries store no bucket.

F Separate Extended-Budget Trials

Extended-budget trials. Table A3 separates single-trial populations on the same Xeon/JAR: 5 comparable Direct-Full decisions match fixed-budget Lazy; 3 Travel (4, 8) WIN decisions resolve prior timeout. Direct-Full timeouts, Surveillance/R1 heap exhaustion and Lazy PC2/R2 timeout persist at expanded heaps. The shared Travel (6, 1) timeout triggers the prespecified (6, 4) skip (records in S4).

Table A3: Separate enlarged-budget single trials (7,200 s cap). S4.6/S4.10 retain every run, failure and skip.

Population	Heap (GiB)	Planned	WIN	LOSS	TO	OOM	Skip
Earlier (four methods)	64/200	27	7	1	16	1	2
Eager	200	10	3	1	5	1	—

On the same Xeon/JAR, Eager’s 200 GiB/7,200 s single trials return 3 WIN where Direct-Full remained unresolved at that heap, discovering about $(1.08\text{--}2.66) \times 10^7$ states with UC pruning (60–68,000 times

Lazy); Industry/R1 returns LOSS with 94,046 states (2.4 times). All 4 decisions match fixed-budget Lazy; ratios concern decided pairs only. The other 6 trials yield 5 TO/1 OOM at about 209–212 GiB peak RSS, a process-memory measure (S4).

G NEW and UPD History-Scope Derivations

History-scope checks. Six target/quiescence fixtures match raw-LTS reconstruction. All 138 lookup exports match 1,587 reconstructed entries (900 non-error, 687 ERROR). ERROR entries, missing signatures and unknown states are outside the partial initializer domain; only retained non-error entries enable initialization. These are functional checks, excluded from performance statistics.

For new requirements (NEW), independent monitor/observer exploration checks all 138 occurrences under the declared history scopes. Equal fluent vectors have equal continuation languages and match frontend lookup; every admitted safe history reaches such a pair, proving RS inclusion. Exactness also needs nonempty actual safe histories, still unverified.

For interval requirements (UPD), a fresh reference starts at entry, consumes its marker once and uses actual fluent values. The 89 R2 initializers use only update-stage fluents whose commands cannot precede entry. The 46 R1 formulas have $\Box(G \Rightarrow \neg A)$: G is the command-controlled interval guard and A a disjunction of ordinary-event predicates. The guard starts false and changes only on excluded pre-entry commands; the action predicates reset at entry. Static formula, label, reset and DFA checks therefore establish exact entry-scoped RS for all 135 UPD requirement occurrences (derivations in S3–S4). Execution assumptions remain unvalidated.

For R1, the general source/export condition checks 46 occurrences, 4,722 state–label pairs and 854 entry valuations; seven negative mutations are rejected (S3). The older E' condition, restricted to update-stage fluents, left these ordinary-action predicates unverified; those results are retained. GSM/R1’s concrete derivation is in Technical Appendix M.

H Proof of Conditional Trace Lifting

Linking the phases. The linked model connects disjoint old/update/new phases by entry $z \mapsto \text{Init}(z)$, policy edges Succ_σ and handover $q \mapsto \kappa(q)$. Here $\text{Init}(z)$ is the admitted update state copied and initialized from old state z , Succ_σ gives the policy successors, and CL^v is the monitored closed loop of endpoint version v . Entry and handover add no game event. Let α be a phase-disjoint observation map.

A1 (endpoints): actual endpoint observations map to same-labeled edges of CL^v and preserve its component and monitor states.

A2 (update observations and progress): at a non-goal, filtering the complete environment event set disables every controllable event outside $\sigma(q)$ and disables no uncontrollable event; every remaining event/outcome maps into $\text{Post}(q, a)$ and is reported with its successor identity. The resulting set is nonempty and a next event is eventually observed. This progress premise assumes no fairness among event choices.

A3 (commands and observation): each command starts at uncontrollable quiescence, finishes atomically and failure-free in finite time, and reports one modeled transfer or monitor-lifecycle transition with its successor identity. Internal τ steps, unobservable outcomes and divergence are outside the model.

A4 (boundaries): entry atomically observes the actual reachable old state z and switches to $\text{Init}(z)$; at a quiescent goal, ordinary controllables are disabled and handover installs its selected controller state and switches control. Each boundary is a finite, failure-free linearizable operation preserving the specified component and monitor states [14].

Theorem H.1 (Conditional trace lifting). *For a returned WIN certificate $(\sigma, \text{rk}, \kappa)$ of an eligible contract, every execution satisfying A1–A4 maps through α to the linked model. From an observed entry $\alpha(x_0) =$*

$\text{Init}(z) \in Q_0$, the update reaches quiescent handover within $\text{rk}(\text{Init}(z))$ observed game events and then hands control to the compatible endpoint. With complete activation histories as in Section 5.1 and RS for the supplied initializers, each declared active interval satisfies its requirement/residual interpretation. Agreement with application intent remains an input obligation.

Proof. Fix an execution satisfying A1–A4 and a returned certificate $(\sigma, \text{rk}, \kappa)$. We use the phase-disjoint observation map α from the theorem; the assumptions require it to preserve the specified component and monitor states, not merely event names. A1 maps the old prefix to same-labeled edges in the monitored old closed loop. At entry, A4 supplies one linearization point at which the actual reachable old state z is observed and control changes to $\text{Init}(z) \in Q_0$. Entry contributes the linked model’s boundary edge and no game event. Thus the update observation starts in the certificate domain.

Suppose the current observation is a non-goal certificate state q . A2 filters controls according to $\sigma(q)$ while retaining every enabled UC event. Its next reported event and successor identity therefore give an edge $q \xrightarrow{a} q'$ in the complete policy successor relation. For a command, A3 additionally supplies the required UC quiescence, finite atomic completion and exact modeled successor. Ordinary reports satisfy A2 directly. The checked WIN certificate includes every such successor, so q' remains safe and in its domain, with $\text{rk}(q') < \text{rk}(q)$. This establishes the update-prefix simulation and rank invariant by induction, for every outcome rather than a selected favorable one.

A2 makes a next event eventually observable at every non-goal; A3 rules out a nonterminating command between observations. Consequently the execution cannot stop or stall at a non-goal. An infinite sequence of update events would give an infinite strictly descending sequence of natural ranks, which is impossible. At most $\text{rk}(\text{Init}(z))$ game events therefore reach a goal. This is an event bound: the assumptions provide no uniform bound on the time between observations. They also impose no fairness on which enabled event or outcome occurs.

The goal predicate supplies UC quiescence and a compatible reachable new-endpoint state $\kappa(q)$. A4 disables ordinary controls, preserves the specified component and monitor states, and completes handover at a finite linearization point. Representing this operation by the linked boundary edge adds no game event. A1 then maps every new-phase observation to the selected monitored endpoint’s same-labeled continuation. Thus the complete observed execution maps into the linked model. The argument assumes the two boundary implementations have the A4 properties; it does not prove their linearizability or atomicity from implementation code.

Finally, certificate safety preserves each supplied monitor while it is active. For each actual activation history, completeness places that history in the declared scope and RS includes the monitor’s accepted continuations in the history’s legal residual language. Applying this inclusion to every active interval gives the claimed requirement interpretation. The choice and intended meaning of those scopes remain input obligations. Unobservable transitions, divergence, failed commands and implementations violating A1–A4 are outside the theorem. $\square$

I Policy Requirement-Boundary Argument

Opposing requirement boundaries. Policy’s audits must overlap, but its old/new roles must not overlap. No precedence is supplied: the monitors force new-audit start before old-audit stop and old-role stop before new-role start. The saved policy interleaves these boundaries and its identity transfer; from the logged entry, service first clears the log. Grouping both starts makes the first global start overlap roles; grouping both stops makes the first global stop remove audit coverage. Both merged entries therefore lose, and the saved losing closure checks all alternatives. Ordinary events can be withheld, so this contract does not promise service continuity. The artifact gives its complete monitor/initializer/endpoint data.

A finite boundary witness. Policy(2) preserves audit coverage and excludes simultaneous old/new roles (Table A4). Here o, n mean old/new, r running, u, l unlogged/logged, A, R audit/role, and ℓ_z, a_z mean log/access for version z .

Table A4: Complete finite-game input for Policy(2) under Section 3’s rules. All events are controllable. Unlisted monitor transitions self-loop, including at absorbing error err; unlisted plant/controller edges are absent. Every monitor has alphabet Σ .

Part	Finite definition
Plant and transfer	Both versions have sole, initial state r , self-looping on $\Sigma_N = \{\ell_o, \ell_n, \text{service}, a_o, a_n\}$. Transfer ρ maps r to r and changes only its version tag.
Controllers	For $z \in \{o, n\}$, C^z has alphabet Σ_N , initial state u , edges $u \xrightarrow{\ell_z} l$, $l \xrightarrow{\text{service}} u$, and a_z loops at u, l . An unreachable third state has all Σ_N loops (the saved alphabet-padding state).
Endpoint monitors	Old: A_o, R_o ; new: A_n, R_n . A_z has states u, l, err , initially u , and edges $u \xrightarrow{\ell_z} l$, $l \xrightarrow{\text{service}} u$, $u \xrightarrow{\text{service}} \text{err}$. R_z has states ok, err , initially ok , with $\text{ok} \xrightarrow{a_z} \text{err}$ ($\bar{z}$ is the other version).
Interval monitor	States are $\{b \in \{0, 1\}^4 : (b_1 \vee b_2) \wedge \neg(b_3 \wedge b_4)\} \cup \{\text{err}\}$, initially 1010. Bits track active (A_o, A_n, R_o, R_n). Commands $\text{stop}_A, \text{start}_A, \text{stop}_R, \text{start}_R$ set $b_1 = 0, b_2 = 1, b_3 = 0, b_4 = 1$, respectively; unsafe results enter err. Ordinary events and ρ preserve bits.
Commands and initializers	The four boundary commands and ρ occur once each; precedence is empty. Stops deactivate old monitors; starts activate new ones. At either (o, r) or (n, r) , initialize $A_n = u, R_n = \text{ok}$; the interval initializer is defined only at (o, r) , returning 1010. These domains are complete.
Entries and targets	For $x \in \{u, l\}$, entry q_x admits $((o, r), C^o = x, A_o = x, R_o = \text{ok})$, discards C^o , installs interval state 1010, leaves new monitors inactive and marks all commands pending. Both $z_x = ((n, r), C^n = x, A_n = x, R_n = \text{ok})$ are targets. Goals require safety, no pending commands and a matching target; UC quiescence is automatic.
Boundary merge	One global start activates both new monitors; one global stop deactivates both old ones. Each monitor composes the two member transitions, audit then role. All other inputs remain fixed.

Lazy returns from q_u $\text{start}_A, \text{stop}_A, \text{stop}_R, \rho, \text{start}_R$, traversing interval states 1010, 1110, 0110, 0100, 0100, 0101, then reaches z_u . From q_l , service first reaches q_u . Ranks five/six bound completion, not shortest length; this safe order is not unique. Audit overlap and a role-free interval are permitted.

Both merged entries lose: the first global start overlaps roles; the first global stop removes audit coverage. Either enters err; ordinary events and transfer preserve these bits. Saved losing closure checks all alternatives, and merge validation confirms equal effects in both member orders.

These supplied games compare boundary granularity, without a DUCS expressiveness claim. Ordinary events can be withheld; service continuity is not required. Actual-history completeness, application adequacy and A1–A4 remain unestablished.

J Separate Evaluation Populations and All-Contract Grid

The following descriptions preserve the separate secondary populations reported in the main paper. They are not pooled with the fixed-budget comparisons. Figure A2 displays all 108 fixed-budget outcomes. Figure 5 in the main paper gives absolute costs for every completed Lazy/Direct-Full pair.

	Lazy			Update-first			Eager			Direct-Full		
Adapted input	B	R1	R2	B	R1	R2	B	R1	R2	B	R1	R2
GSM	W	W	W	W	W	W	W	W	W	W	W	W
Industry	W	L	W	W	L	W	W	T	W	W	T	W
MetaSocket	W	W	W	W	W	W	W	W	W	W	W	W
PowerPlant	W	W	W	W	W	W	W	W	W	W	W	W
PC1	W	W	W	W	W	W	W	W	W	W	T	W
PC2	W	W	T	T	T	T	T	T	T	T	T	T
Railcab	W	W	W	W	W	W	T	T	W	T	T	W
Surveillance	W	W	W	W	W	W	T	T	T	T	T	T
Workflow	W	W	W	W	W	W	W	T	W	T	T	T

W: WIN L: LOSS T: timeout B: Base; R1/R2: interval variants

Figure A2: Every fixed-budget contract–method outcome. Each tile is one of the same 9 inputs $\times$ three requirement variants, under a 1,200 s cap and 64 GiB heap. WIN and LOSS are completed decisions; timeout leaves feasibility unresolved. The methods change exploration, keeping entries, successors and goals fixed. The comparison evaluates complete exploration procedures. Workflow/R2’s Direct-Full tile uses the recorded same-setting timeout after interruption; both attempts are retained.

Base/R1/R2’s 5/3/6 pairs give median state ratios 111.89/11.88/2.75 and solver ratios 40.02/3.12/9.14. The largest state savings are in Base, but different completed populations do not isolate interval requirements’ effect.

J.1 Full+UC observations

Full+UC yields 16 WIN/1 LOSS/10 TO on Mac, 15 WIN/12 TO on Xeon; Industry/R1 times out on Xeon despite Mac’s LOSS. State counts equal Eager in 14/16 cells, differing on GSM/R1 and PowerPlant/R1. On PC1/Base, Direct-Full/Full+UC/Eager each discover 407019 states versus Lazy’s 83. Full+UC, Eager and Lazy all use UC pruning. Builds, hosts and repetitions differ, precluding single-factor timing claims (all rows/conditions in S4.7).

J.2 Legacy reference

Separate Legacy reference. Legacy uses our MTSA fork’s traditional DUCS/GR backend with a subsequent-control objective, differing from FG’s fixed endpoints. It yields 9 matching WIN, 13 OOM and 5 unmeasured snapshot-cap cases. At 200 GiB/3600 s, 18 single trials give 11 OOM and 7 instrumentation stops, no decisions. S4 retains build identities, source/fork discrepancies and enlarged-heap records; these failures prove no impossibility.

J.3 Controlled, Travel and extended-budget populations

Controlled families vary fanout, transfer outcomes, UC events/outcomes, requirements or precedence. All methods agree on 31 WIN/two LOSS across 33 inputs (132 single-run jobs); multiple graph measures may vary together (definitions/counts in S4).

Travel couples an agency with N amenities, K UC choices and $0-K-1$ selection steps: size, branching and length covary. Adaptations change purchasing, admission, transfers and monitors without claimed trace equivalence (S4). Across 192 first trials on 48 settings, each OTF variant yields 20 WIN/26 TO/2 OOM, Direct-Full 18 WIN/28 TO/2 OOM. Only $(N, K) = (4, 4), (4, 6)$ extend OTF’s decided range. Both approaches still time out on $(6, 1)$ at 200 GiB/7,200 s, so the original heap cap is not the sole limit. S4 retains every setting and the 18 completed-pair state/time comparisons.

Extended-budget limits. At 7,200 s and heaps up to 200 GiB, 27 earlier jobs include 16 TO/1 OOM/2 skips; 10 Eager jobs include 5 TO/1 OOM. Comparable decisions agree with fixed-budget Lazy. The separate extended-budget OTF decisions on Travel $(4, 8)$ agree with one another, but fixed-budget

Lazy timed out there, so they are not baseline decision matches. These single trials are excluded from its medians; all outcomes remain in the artifact appendix and S4.

J.4 Additional granularity witnesses

The boundary fixture requires alignment after old-stop and before new-start: the requirement-block restriction forbids that intervening ordinary event and loses, whereas boundary merging merely re-names the single stop and single start and remains WIN. Cell_n has one workpiece circulating through n stations with empty-only transfers; its single old stop and new start make boundary merging a re-naming. It remains WIN with merged boundaries and LOSS with both merges. Restricting transfers to initial local states yields 0 fine WIN/merged LOSS, 6 both-LOSS and 2 both-WIN pairs, with 10/18 incomplete; enabled UC requests can obstruct transfer quiescence.

Eight original finite files produce ten WIN/four LOSS across fourteen jobs covering cell transfer, observed branches, requirement interleaving and neutral controls (parameters in S1). S4 retains excluded populations and earlier failed inputs. The alignment sequence states a required boundary order, not the complete saved Java execution trace.

J.5 Application policy and losing-contract details

Following an application policy. Railcab Base retains both transfer outcomes and their distinct continuations, memories and targets. R1's braking ban causes earlier braking or completion without it; crossing requires approach without a last emergency brake. Both all-entry exports pass certificate/Link checks, with no timing/runtime validation (complete tuples/branches in S4).

What a losing contract means. Industry has three validation components: gate-form GF1, TOR and DSD1. Industry/R1 loses at 42 of 131 entries by independent exhaustive checking. A reachable safe quiescent state has no supplied target: validating GF1 or TOR violates the new requirements; the remaining safe alternative, DSD1, admits a rejection cycle, while disabling it deadlocks. The full losing fixed point, not this one path, excludes every update order. R1 forbids validation between old stop and new start. DUCS instead synthesizes continued new-specification control [25, Defs. 4.1–4.3]; its original build and aligned fork win on the source model. FG LOSS depends on supplied initializers, requirements and targets. Exempting GF1 from R1 makes all 131 entries WIN via earlier paths before the DSD1/TOR alternatives are activated, not escape from the displayed trap; this separate Mac contract variant supplies no timing data.

J.6 Input adaptation details

Nine inputs manually adapt eight DUCS sources; both ProductionCell sizes share a parent. Base/R1/R2 vary interval requirements, fixing components, transfers, endpoints and old/new goals (Technical Appendix C). Adaptations make PC tool results UC, revise Surveillance battery behavior/safety, order GSM/R2's decode boundaries before ENV transfer, shorten Railcab transfer paths while keeping both outcomes, and omit PowerPlant's initial ENV transfer pair. Recorded provenance does not establish whole-problem equivalence.

J.7 Independent-update construction

For $n \geq 1$, take n components, each with one old and one new state and a singleton transfer. Every version and its endpoint controller admit a controllable *tick* self-loop, making both endpoints deadlock-free. There are no monitors, dependencies or uncontrollables; the unique new endpoint is available for handover. These inputs are eligible. Each update state is the subset of transferred components, and all 2^n subsets are reachable; Direct-Full discovers them all.

Under Lazy's default order, pending transfers precede ordinary events in this family, so expansion follows one n -step chain. Because Open precedes Retry, it reaches the sole goal after discovering exactly

$n + 1$ states. Backward propagation then assigns ranks $0, \dots, n$ and proves the entry state before any alternative query. The ordinary self-loops are losing choices under finite completion; both methods use the same terminal goal. Thus the separation concerns exploration of one supplied game.

J.8 Memory and returned policies

Lazy’s five-run median peak working sets span 0.102–7.36 GiB (median 0.837), including JVM/native overhead; Eager/Update-first sometimes use less. Its 25 WIN policies contain 8–1,071 states (median 55) and maximum ranks 4–34 (median 19). Ranks bound events, not downtime or optimal length. Counts match all five repeats; LOSS/TO has no winning policy.

J.9 Separate Performance Populations

The 12 capped Direct-Full attempts on Lazy-decided contracts give total-JVM lower bounds against Lazy medians, excluding PC2/R2 (both unresolved). These are single attempts, not solver ratios or medians; S4.5 retains all 27 absolute times/outcomes.

Full+UC matches Eager’s state counts in 14/16 cells; state-set equality is unchecked. OTF without pruning is unimplemented. Different builds, hosts and repetitions prevent single-factor timing claims. Technical Appendix J also retains Legacy’s subsequent-control reference and unresolved outcomes.

Controlled families agree on 31 WIN/two LOSS over 33 inputs, without full factor isolation. Travel covaries agency/amenity size, branching and length: OTF gives 20 WIN/26 TO/2 OOM per method; Direct-Full gives 18 WIN/28 TO/2 OOM. Both time out on (6, 1) at 200 GiB/7,200 s. Technical Appendix J retains extended TO/OOM/skips and the extended WIN versus fixed timeout at (4, 8); extended trials never enter medians.

J.10 Finite contracts for the constructed families

These definitions specify the inputs behind Table 4 and the Rolling grid. They describe finite models, not implementations of deployment platforms. Component versions have distinct tags even when their transition graphs coincide. Unlisted plant and endpoint-controller edges are absent; unlisted monitor labels stutter, and errors are absorbing. Ordinary events are controllable unless explicitly called UC. There is no explicit command precedence. OLD/NEW monitors are empty except in Rolling+Audit; the stated interval monitors apply until handover. The fixed endpoint graphs determine every reachable old entry and the listed loadable new targets. The main paper defines quiescence, command execution and the compared merges.

Rolling(n, m) and block size k . For $2 \leq n \leq 6$, $1 \leq m < n$, each old component has one ready state with a $serve_i$ loop. Its new version has booting and ready states, with UC $ready_i$ from booting to ready and $serve_i$ only at ready. A local transfer maps old ready to new booting. The interval monitor starts at $r = n$, decrements r by the transferred block size, increments it on $ready_i$ (capped at n), and errors below m . Both fixed endpoints are all-ready tuples whose one-state controllers admit all $serve_i$ loops; these supply one entry and one target. Consecutive blocks of at most k , including the final remainder, are transferred atomically. The 15 (n, m) pairs compare $k = 1$ with $k = n$; the separate 70-cell grid varies $1 \leq k \leq n$. Proposition 6.2 proves the threshold for these inputs.

Canary(n, m). The same 15 (n, m) values use one old healthy state H and new states H, H_P, B_P, B . Only H serves. A transfer has two adversarial outcomes, H_P and B_P ; UC reports take H_P to H and B_P to B . A controllable restart takes B to H . The interval monitor is a bit vector initially zero: a broken report sets bit i , a restart clears it, and more than $n - m$ set bits cause error. Other labels stutter. This is a *reported-broken* budget; pending physical states are not identified with already reported failures. The saved certificate also checks physical health separately. The all-healthy old/new tuples and one-state serve-only endpoint controllers give one entry and one target. Merging transfers synchronizes

their commands and retains every Cartesian-product outcome. The healthy-outcomes-only and no-recovery variants restrict transfer images or remove restart, respectively; their unchanged both-WIN and both-LOSS results delimit the recovery mechanism.

Canary certificate scope. The existing separate check passed on every state of all 30 returned WIN certificates: the 15 parameter pairs under Lazy and Direct-Full. It counts replicas in H or H_P against the threshold m ; B_P and B do not count. Thus it checks model-state health, while H_P still cannot serve. It does not check continuous service, every explored state or runtime behavior. The other 30 saved checks concern LOSS mechanisms, not physical-health success. Fine and merged transfers share capacity reasoning with Rolling, but their reported-health contracts are different; no safety-preserving reduction to the Rolling game is claimed.

DB-Rolling($3, m$). Here $m \in \{2, 3\}$. Each old component has primary P , secondary S , and election-pending E states; the new version adds booting B . Serving loops exist at P, S, E . A controllable $stepdown_{i,j}$ synchronously changes P_i to S_i and S_j to E_j ; UC $elect_j$ changes E_j to P_j . A transfer maps only S to B , and UC $ready_i$ changes B to S . The interval budget starts at three, decreases by the transferred block size, increases by one on readiness (capped at three), and errors below m ; election does not consume this nonbooting budget. The old endpoint is (P, S, S) , the new target (S, P, S) , both with serve-only controllers; unreachable padding retains the remaining ordinary alphabet. Thus the contract specifies a change of primary. With $m = 2$, a fine policy can change the primary and transfer secondary components successively. The merged command requires all-secondary states, which cannot be quiescent while an election remains pending. With $m = 3$, even the first fine transfer violates the budget. These models do not assert uninterrupted write service.

Rolling+Audit. This is Rolling($2, 1$) with controllable log_{old} and log_{new} loops in every local plant state. Each version's audit monitor has unlogged, logged and error states. Its own log sets logged; either serve changes logged to unlogged but errors from unlogged. The other log and all remaining labels stutter. The new initializer selects unlogged on all nine tagged physical tuples. The interval state is (r, o, a) , initially $(2, 1, 0)$: transfers/readiness change r as above, old-stop clears o , and new-start sets a . Safety requires $r \geq 1$ and $o \vee a$, enforcing overlap of the requirements. Each fixed all-ready endpoint controller alternates its own log with either serve; both its unlogged and logged states are entries/targets, and unreachable controller padding retains the remaining ordinary alphabet. These specify the memories, initializer and coverage obligation used in the comparison. Merging the two transfers loses; the single old-stop and new-start cannot be combined further, so merging requirement groups alone is only a renaming.

Threads(n, B). For $2 \leq n \leq 6$ and queue capacity $B \in \{1, 2\}$, each worker is idle or busy; component 1 additionally stores queue length $0 \leq q \leq B$. UC arrival increments q below capacity. At capacity, backpressure disables arrival whereas saturated arrival is a self-loop. Controllable $dispatch_i$ requires $q > 0$ and worker i idle, decrements q and makes that worker busy; for $i > 1$ it synchronizes with component 1. UC $finish_i$ returns a busy worker to idle. Old/new plant graphs coincide. Each local transfer is identity on idle states and preserves component 1's queue. There are no monitors. One-state endpoint controllers admit all ordinary labels; their reachable products from the empty, all-idle initial tuple give all old entries and all new targets. These two arrival modes yield 20 pairs. Under backpressure, disabling dispatch lets the finite queue fill and busy workers finish, permitting globally quiescent transfer. Saturated arrival can persist forever and prevents even fine transfer under the contract's quiescence rule. The negative control therefore distinguishes a genuine transfer opportunity from an implicit fairness assumption; it is not a reproduction of a multithreaded runtime.

K Complete Native DUCS Construction for Cell

Table 2 in the main paper compares the interfaces; the following construction makes that comparison concrete.

Table A5: Contract interfaces and construction responsibilities. Native DUCS permits stages explicitly encoded in its new environment [25, Defs. 4.1–4.3, Sec. 4.2].

Aspect	Native DUCS	FG-DUCS
Transfer input	Old/new environments and global relation	Component pairs and local relations g_i
Requirement	Old/new goals, transition requirement and fluents	Old/new/interval monitors, initializers and precedence
input		
Constructed structure	Interrupt composition and update protocol; extra stages supplied in the new environment	Mixed versions, command guards, lifetimes and all local outcomes
Controller choice	Old controller supplied; update and subsequent control synthesized	Both controllers and targets supplied; update policy and load state selected
Returned result	Updated controller and state relation, or no solution	Policy, rank and handover, or losing region

Both require justified models. Explicit native stages, guards and fluents preserve both Cell paths and their subsequent traces. This is one-direction witness preservation, not general equivalence or measured effort. Fixed new control excludes solutions requiring a different continuation.

Complete construction and witness-preservation proof for main Section 3.4. We construct a correct Cell update using native DUCS environments, relation R , transition requirement T and boundary protocol O [25, Defs. 4.1–4.3 and 5.1].

DUCS’s `hotSwap` event requests the update and interrupts old control. Its `reconfigure` milestone switches environments; our extra h connects mixed stages to a fixed endpoint.

Environments and entry. Refine the old controller to its reachable closed loop $c_A \xrightarrow{m_B} c_B \xrightarrow{m_A} c_A$, preserving ordinary traces and recording the holder. The old environment has these same moves. Let E' contain eight stages $(v_A, v_B, L) \in \{o, n\}^2 \times \{A, B\}$ and three post-handover states. In a stage, m_B moves A to B , m_A moves B to A , and j self-loops only if $v_B = n, L = B$. Transfer ρ_i changes $v_i = o$ to n only while station i is empty. A controllable h maps (n, n, L) to z_L . Post-handover edges copy the fixed new closed loop:

$$z_A \xrightarrow{m_B} y_B \xrightarrow{j} z_B, \quad z_B \xrightarrow{j} z_B, \quad z_B \xrightarrow{m_A} z_A.$$

Here y_B is inspection-pending; z_A, z_B are clear targets. The standalone initial state is z_A , but interrupt entry uses $R : L \mapsto (o, o, L)$. The 11-state count excludes protocol and monitors and is not a minimality result.

Requirements and memory. Write $s = \text{startNewSpec}$ and $t = \text{stopOldSpec}$. Protocol O accepts these physical stutters. Let $\dot{a}$ record last event a , resetting on every other label. Fluents $B = \langle m_B, m_A, \text{false} \rangle$ and $P = \langle m_B, j, \text{false} \rangle$ retain holder and uninspected-arrival history. Take $G = \neg \dot{j}$ and $G' = \neg(P \wedge \dot{m}_A)$ under native old-until-stop and new-after-start scopes.

For each event a , let S_a be its initially false, never-reset occurrence flag. Take T to be the conjunction

$$\begin{aligned} &\Box(\dot{s} \Rightarrow S_{\text{reconfigure}}), & \Box(\dot{t} \Rightarrow S_s), \\ &\Box(\dot{h} \Rightarrow S_{\rho_A} \wedge S_{\rho_B} \wedge S_s \wedge S_t \wedge \neg(B \wedge P)), & \Box(\text{reconfigure} \Rightarrow \Diamond \dot{h}). \end{aligned}$$

Interrupts, O , version guards and the post-handover region ensure at most one occurrence of each update event. This analytical T includes eventual h , beyond the published safety- T implementation [25, Sec. 5.3]; no solver experiment is claimed. Native DUCS still requires eventual `stopOldSpec`, `reconfigure`

and startNewSpec, including in its safety- T implementation. Here reconfigure enters the mixed stages; h is an additional fixed-target connection, so those three milestones do not by themselves require h . We do not rule out another encoding within safety- T .

Proposition K.1 (Cell witness preservation). *This staged native DUCS instance admits a correct update from both reachable old entries. On the segment from hotSwap through h , erasing these two events and reconfigure and renaming s, t to Cell’s boundaries gives the displayed FG policy paths. After h , its ordinary traces equal those of the supplied new closed loop rooted at the selected target.*

Proof. Use DUCS’s assembled environment, including O , its old-event renaming/fluent extension, and controller extraction [25, Sec. 5, Theorem 5.1]. Preserve old behavior before hotSwap. Map $H(c_L)$ to the corresponding pre-reconfiguration controller state; then choose reconfigure, the six-/five-event path of Figure 2, h , and every fixed-endpoint edge thereafter. All choices are enabled in the assembled environment; no ordinary UC event remains after initiation, ensuring legality and deadlock freedom.

Old j is disabled until t . On safe active prefixes the correspondence is: the FG monitor is pending iff $B \wedge P$. At s , a B holder has an earlier m_B and no j , so both are pending. At an A holder, B is false and FG is clear, although P may remain true from an earlier visit to B . This is equivalence of permitted continuations on the plant, not equality between P and the monitor state: m_A and j are disabled at A , and the next m_B makes both representations pending. Thereafter j clears P and the monitor. An enabled m_A comes from B ; it leaves P unchanged while setting $\dot{m}_A$, so it violates G' exactly when FG was pending. Otherwise it leaves $B \wedge P$ and FG clear. Transfers and boundaries preserve the correspondence. Thus every ordinary extension has the same safety outcome in both representations. The finite paths satisfy the boundary order and reach all-new, clear targets, fulfilling T and the native update milestones. After h , the synchronized environment/controller copies retain every fixed-endpoint edge. Their reachable product is diagonal, giving the claimed ordinary-trace equality. $\square$

The native encoding explicitly supplies stages, guards, holder refinement, history fluents, target gate and post-handover graph. FG-DUCS generates the update-game counterparts from local transfers, lifecycle rules, initializers and fixed targets, returning rank and handover certificates. This deterministic, fully controllable example establishes one-direction witness preservation, not a general translation, solution-set equivalence, measured effort saving or performance advantage.

Table A6: The same Cell obligations through the two interfaces. Native entries refer to this explicit construction; they are not lower bounds on other encodings. FG-DUCS supplies the generated structure in addition to the modeler’s inputs.

Obligation	Explicit native Cell modeling	FG-DUCS input and generated structure
Empty-station re- placement	Eight mixed stages in E' , holder moves and version/empty-station guards for ρ_A, ρ_B .	Old/new models for A, B and $g_A = g_B = \{(e, e)\}$ generate mixed versions and transfer edges with lifecycle/quiescence guards.
Inspection life- time	Holder-refined old control; $B, P, \dot{a}, S_a$ fluents and G, G', T encode memory and boundaries.	Typed monitors, initializers and declared precedence generate boundary events, active-memory updates and initialization under the type-defined lifetimes.
Fixed continua- tion	Post-handover graph in E' , a guarded h and eventual- h in T .	Fixed controllers and load targets generate matching/goal conditions; synthesis selects a load state and handover map.

Native DUCS also constructs its standard protocol O and interrupt composition; these are not additional modeler inputs.
The native witness retains every fixed-endpoint edge after h .

L Saved PC2 Activation Histories and Their Boundary

The supplied old controller and pre-synthesized new controller are fixed. Baseline PC2’s old requirements and 12 new requirements remain; added new-endpoint calibration availability yields 13 new starts, separate from the interval duty. Transfers map old destinations to calibration counterparts, retaining ordinary observer memory. Work, calibration requests and updates are controllable; arrivals, tool outcomes, resets and calibration completion are UC. Entry inherits old memory and initializes the interval monitor; new-start looks up the current ordinary observer tuple. No precedence is supplied. Targets match components and new-monitor memory at reachable new-endpoint states.

Interval R_{CAL_READY} observes transfers as calibration entry; endpoint availability observes ordinary requests. Completion restores both. Handover matches component, observer and new-monitor memory; real-history and runtime obligations remain unvalidated.

Table A7: One saved 31-event PC2-Rolling path. O/N: old/new; ARM: unprocessed workpiece; CAL: its calibration counterpart. Ordinary observers stay zero.

Reached after	Arm 1	Arm 2	Available	Rank
Update entry	O/initial	O/initial	2	31
Two arrivals, twelve old stops	O/ARM	O/ARM	2	17
Transfer arm 1	N/CAL	O/ARM	1	16
Calibration completion 1	N/ARM	O/ARM	2	15
Transfer arm 2	N/ARM	N/CAL	1	14
Calibration completion 2	N/ARM	N/ARM	2	13
Thirteen new starts: handover	N/ARM	N/ARM	2	0

At handover, all 13 new monitors are 0 and no command remains; the matched endpoint has both arms operational. Certificate identifiers are in Technical Appendix L. This path supplies no timing and does not replace all-entry/all-outcome checking.

This analysis traverses existing PC2-Rolling exports; it runs no solver and adds no timing trial. The selected requirement is $\Box(out_1 \Rightarrow Drilled_1 \wedge Cleaned_1 \wedge Painted_1)$. The three fluents start false, successful tool events set them, and $reset_1$ clears them. An out_1 with any false fluent violates the requirement. Other update commands stutter for these fluents.

The saved old closed loop contains 126 reachable states and 348 edges; the policy contains 381 states and 522 edges. Matching component states, old-monitor memory and ordinary observers identifies each policy entry with one old state. Every maximal policy run crosses the single selected start edge $q_{13} \rightarrow q_{12}$, which installs monitor state 0 at two NEW/ARM components with all 24 ordinary observers zero. Ranks strictly decrease; removing that start edge makes the unique goal unreachable from every entry. The main paper’s displayed path starts at q_{126} and ends at q_7 , whose handover target is new-00000009. All 13 new monitors are 0 there and no command remains. Its prefix to q_{12} has two arrivals, twelve stops, four transfer/completion events and eight starts, with no out_1 or successful tool result.

For each entry, find an old prefix from the declared old initial state and a policy prefix ending at this start. Keep an edge only if the source fluent values and its label do not violate the selected new requirement. Forward reachability in the old graph and reverse reachability from the start in the policy graph yield the table. Separate per-entry breadth-first traversals construct positive witnesses or record failure of each portion; set-based fixed points reproduce the same partition.

Safe old prefix exists	Safe policy prefix exists	Entry count
Yes	Yes	39
No	Yes	21
Yes	No	66

All 126 entries reach the start through some unrestricted saved-model history. The 39 positive rows

establish existential nonemptiness at this one reached tuple, not safety of all histories. An old cycle

$$in_1; drill_1; drillOk_1; polish_1; polishOk_1; clean_1; cleanOk_1; out_1; reset_1$$

returns to the identical old initial tuple. It violates the new requirement at out_1 because $Painted_1$ is false. Prepending this cycle to any entry's history preserves its final activation tuple while retaining a past violation. The new monitor is absent in every old state and all 375 policy ancestors of its start, so none of these negative histories refutes its active-lifetime safety. They show why activation-safe and history-inclusive scopes cannot be conflated.

Replay of source-declared plant moves and all 24 observer updates agrees with all 348 old and 522 policy edges. This establishes consistency of these saved histories, not independent frontend correctness or completeness of exported outcomes. The artifact retains all 126 rows, positive paths, negative reasons and 18 boundary edges separating the safe reachability sets. Only one reached tuple of one requirement is covered; the reported 38,288 initializer guard tuples, the other 12 PC2 new requirements and the 138-occurrence population are not checked for nonemptiness by this analysis.

Direct ACT argument after the selected start. ACT begins the invariant with the actual observer values at activation. For the selected requirement these values are all zero. The saved policy graph's complete suffix is:

Source (rank)	New requirement started	Target (rank)
q_{12} (5)	OUT_IF_FINISHED_2	q_{11} (4)
q_{11} (4)	TOOL_ORDER_1	q_{10} (3)
q_{10} (3)	TOOL_ORDER_2	q_9 (2)
q_9 (2)	PAINT_ONCE_1	q_8 (1)
q_8 (1)	PAINT_ONCE_2	q_7 (0)

All six states have two NEW/ARM components, 24 zero observers and selected-monitor state 0. No ordinary event, transfer or uncontrollable edge occurs in this suffix. The unique goal matches new-00000009. Consequently the source invariant holds at activation and on every remaining update step; matching the verified new endpoint supplies its post-handover guarantee.

The saved graph is closed over 381 entry-reachable states; all 522 edges decrease a natural rank, and no non-goal deadlocks. Removing the unique selected start leaves 375 reachable states and no goal from any of the 126 entries. Thus every maximal saved-policy run crosses the start and follows this suffix. The static checker `check_saved_suffix.py` reproduces these facts from the existing certificate without invoking synthesis. Completeness of modeled outcomes relies on the stored certificate checker; frontend/compiler correctness and real execution are not independently proved here. The argument concerns one requirement and one reached tuple, not all 138 NEW occurrences. The A/H diagnostics above remain unchanged.

L.1 All New Requirements of the Saved PC2 Policy

The previous history analysis fixes one requirement to distinguish A from H. A separate inspection of the same saved policy establishes the active-lifetime obligation for all thirteen new requirements. It introduces no solver run, input change or timing trial.

The only new-start edges are the chain $q_{20} \rightarrow q_{19} \rightarrow \dots \rightarrow q_7$, in this order: stamping avoidance for arms 1 and 2; calibration availability; clean-once for arms 1 and 2; drill-once for arms 1 and 2; output-if-finished for arms 1 and 2; new tool order for arms 1 and 2; and paint-once for arms 1 and 2. The selected start above is the eighth edge. Every state in this chain has two NEW/ARM components, all 24 ordinary observers zero and rank $q - 7$ at q . Each newly installed monitor is 0 and stays 0. There is no ordinary event, transfer or UC edge after the first new-start.

Proposition L.1 (All PC2 active obligations). *Every maximal run of the saved PC2-Rolling policy from any of its 126 entries activates all thirteen new requirements and satisfies each source invariant from its own activation through handover. The matching verified new endpoint preserves these obligations afterward.*

Proof. The saved domain is entry-reachable and closed, every edge strictly decreases a natural rank, and no non-goal deadlocks. Hence every maximal run reaches the unique goal q_7 . Every q_j , $7 \leq j \leq 19$, has only its preceding chain edge as an incoming policy edge, and no entry lies on the chain. Thus every run traverses all thirteen starts.

For each arm, the source invariants forbid stamping; forbid cleaning, drilling or painting an already completed workpiece; require all three completed-tool fluents at output; and constrain tool-pending fluents by the new tool order. At activation all completed-tool and pending fluents are false, and no ordinary action occurs on a start label. Each invariant therefore holds. The calibration invariant is $\Box(\neg \text{Calibrating}_1 \vee \neg \text{Calibrating}_2)$. Its source fluents start false and are set only by ordinary calibration requests. The old plant has no such request, so every old entry retains false values; none occurs in this policy either, while completion clears them. They too are false on the start chain. Start commands preserve these fluents and produce no ordinary action, so all active source invariants continue to hold until q_7 . This goal matches new-00000009, with both components, all ordinary observers and all thirteen new monitors aligned. The supplied verified endpoint and continuation preservation give the post-handover part. $\square$

This covers all new requirements of this one saved policy at their reached activation tuples. It does not prove initializer exactness over every permitted tuple, safe pre-activation histories, all 138 NEW occurrences, compiler correctness or runtime conformance. The 39/87 history partition above is unchanged. PC2 completes calibration before activating any new requirement; its active update suffix contains only starts. The GSM construction below complements it with ordinary behavior under an active new requirement.

The earlier violations have a concrete source distinction: old PC2 requires drilling, polishing and cleaning, whereas the new output rule requires drilling, cleaning and painting. The update contract preserves its interval calibration obligation and whichever old requirements remain active; it begins each new duty at its start. This defines the specified obligation, without establishing that an operator would choose that scope.

M Activation-Scoped Initialization

Obligations that begin at activation. *Activation-scoped* interpretation ACT starts a pure invariant $\Box\varphi$ with the actual post-boundary observer vector, checking φ there and after each later label. It retains factual fluent values, not a past-violation bit. Start follows the declared observer rules and is not consumed twice. This differs from A's safe-history restriction and H's whole-past obligation.

Proposition M.1 (Activation-scoped invariant initialization). *For such an invariant, let a safe reference history end at vector v . If the supplied initialized monitor has the same continuation language as that reference history's residual, it is exact for ACT at every activation with vector v , including activations whose actual histories violated the invariant before its start.*

Proof. The safe reference prefix leaves only current/future invariant obligations. Deterministic observers give the same future valuations from v on every common suffix, proving equality with ACT. Earlier violations in the actual history do not affect those valuations. $\square$

This requires an appropriate initializer, not merely A inclusion. Section 6.2.1 proves one saved requirement's ACT behavior directly; the 138 A checks remain A checks.

A source-derived policy with active ordinary behavior. This is an analytical construction from the supplied GSM_FG.lts and saved frontend lookup, not an exported solver policy or an additional

experiment. The old plant is the cycle

$$\begin{array}{c}
\text{SENDER} \xrightarrow{\text{shift}X} X \xrightarrow{\text{shift}Y} Y \xrightarrow{\text{shift}Z} Z \xrightarrow{\text{encode}} \text{ENCODED} \\
\xrightarrow{\text{send}} \text{SENT} \xrightarrow{\text{receive}} \text{RECEIVED} \\
\xrightarrow{\text{decode.2 or decode.3}} \text{DECODED} \xrightarrow{\text{output}} \text{SENDER}.
\end{array}$$

The new plant inserts $\text{shift}T$ and state T between Z and encode ; the supplied transfer maps identically named old states to new ones. All ordinary events are controllable. The old safety requirement forbids decode.3 ; the new one forbids decode.2 . The R1 interval forbids ten ordinary labels between old-stop and new-start. Its initializer and source interpretation are derived below.

The new requirement's physical observer is initially zero, becomes one on decode.2 , and becomes zero on every other ordinary label. The saved lookup maps observer zero to non-error monitor state zero and observer one to error (outside the safe initializer domain). In monitor state zero, decode.2 enters error; every other label preserves zero. Update commands and transfer preserve the physical observer. In particular, start reads its current value and does not reset it.

Proposition M.2 (GSM activation with ordinary events). *The supplied GSM/R1 contract admits a state-based completing policy from every old entry whose new-decode requirement is active during ordinary decode.3 and output events. At these activations, the supplied new-requirement initializer is exact for the activation-scoped requirement $\Box \neg \text{decode.2}$; the policy reaches a compatible supplied new-controller state within twelve update events.*

Proof. While old-stop is pending, follow the old cycle only until the first RECEIVED state, then select old-stop and new-start consecutively. Select decode.3 , then output , then the identity transfer at SENDER; disable other controllables. No UC event occurs and each selected label has a singleton outcome.

Every old plant state reaches the first RECEIVED in zero to seven edges without taking a decode edge. This prefix preserves the old requirement and the initially false R1 gap guard. At RECEIVED, the previous receive has set the observer to zero; the same holds for an old entry at RECEIVED. Hence the new initializer is defined. Stop and start preserve that observer, and no ordinary event occurs while the R1 gap guard is true. After start, both decode.3 and output preserve monitor state zero. A choice of decode.2 at that same plant state would enter error, so the active requirement constrains an actual ordinary choice.

Following output , the physical state is old SENDER with observer zero. Transfer yields new SENDER with observer zero and new monitor zero, exactly the supplied new endpoint's initial projection; all three commands are consumed. The remaining safe R1 monitor is of interval type and is not matched against an endpoint requirement. The fixed new controller subsequently excludes decode.2 . Its monitor therefore recognizes precisely the safe active suffixes; violations before activation have no bearing on this ACT claim.

Let d be distance along the old cycle to the first RECEIVED. Give pre-stop states rank $d+5$; give the states after stop, start, decode.3 , output and transfer ranks 4, 3, 2, 1, 0, respectively. Every edge strictly decreases rank, with maximum twelve and rank zero at a matching goal. The pending-command set and active monitors distinguish these phases, so no extra history memory is needed. This proves all-entry safety and finite completion. The same event sequence also places both boundaries before transfer, as required by the separately supplied R2 order; no R2 solver result is inferred here. $\square$

An old execution can already have performed decode.2 before the update request. This construction thus does not establish H or safe full histories for A. It establishes a non-vacuous ACT obligation over ordinary update actions and the fixed endpoint continuation, within this supplied model. It does not validate the compiler, application intent or runtime adapter.

From the source interval to an entry guarantee. GSM/R1 requires $\Box(G \Rightarrow \neg A)$: G starts at old-decode stop and ends at new-decode start; A denotes ten decode/encode, communication and shift events. Every old entry has $G = \text{false}$, since no command has occurred. Entry preserves this guard and resets action predicates. Its monitor has guard-false, guard-true and error states: stop sets the guard, start clears it, and A enters error while the guard holds. Installing guard-false therefore gives exactly the fresh E obligation at every old entry.

The source/export check covers this formula, initializer, all 45 state-label pairs and both action-predicate entry values. This establishes declared E meaning; application intent and runtime conformance remain unvalidated. Technical Appendix G retains the full check population and older unverified results.

N Implementation and AI-Assisted Research Details

Algorithm 1 returns and internally checks the losing region L . The finite-JSON runner serializes its members in `certificate.json`; an independent checker reconstructs successors from the input and checks closure. The FSP/application benchmark driver instead records LOSS, region size and phase-mask statistics, without serializing solver membership. Its saved full-game exports support independent region reconstruction, but do not compare the solver’s member list. These output interfaces therefore provide different saved certificate evidence. The fixed RQ3 runs use Windows 10, an Intel Xeon W-2265, 256 GB RAM and Temurin 17.0.20.1. The main paper states the heap, time cap and repetition policy; separate Mac structural checks and extended trials retain their own provenance.

Codex (`gpt-5.5`, `gpt-5.6-sol`, `gpt-6-astra`; September 30–October 2: GPT-6, variant unavailable) assisted design, implementation, tooling, tests, experiments, analysis, proofs and presentation. Claude Fable 5.1 assisted planning, drivers, source checks and drafts. September 30–October 2 editing used Opus 5.5 and ChatGPT Pro/Latest reviews (response model unavailable); Claude Code inherited project context. Authors checked outputs using tests, certificates, oracles and primary sources and retain responsibility; correlated faults remain possible. Reviews were editorial, not human evaluation.